Wearable Gesture Controller: An IMU-Based Edge Device for Gesture-Driven Smart Home Control

Arya Choudhary, Sanika Singh, and Vishad Sharma
School of Computer Engineering, Manipal Institute of Technology, Manipal, India
ICT 3132 Embedded Systems, V Semester B.Tech. (IT), Submission 2
Registration numbers: 240911438, 240911664, and 240911580, respectively

***Abstract*—Wrist-motion input could complement smart-home interfaces for people whose speech, reach, or dexterity makes conventional controls difficult to use. This project proposes an inertial measurement unit (IMU)-to-command architecture and implements a small, hardware-free software demonstration. The executable pipeline generates explicitly synthetic six-channel streams, applies causal filtering and training-only normalization, forms fixed-length windows, and classifies them with a 16-unit gated recurrent unit (GRU). A background class and confidence threshold produce a no-command outcome. Independent recordings are assigned to training, validation, and test partitions before windowing. On 200 held-out synthetic windows, the recorded run obtained 100.0% accuracy and macro-F1 of 1.000. These values show that the code learns its deliberately simple signal generator; they do not estimate real-world gesture accuracy. The report reviews ten recent research papers, identifies an application-specific integration gap, and discusses buffering, operator support, memory, latency, and optional quantization. No MCU deployment, physical appliance control, or power measurement is claimed. Scopus coverage of the reviewed papers remains unverified and requires institutional confirmation.**

***Index Terms*—embedded systems, IMU, gesture recognition, GRU, synthetic data, edge inference, accessibility**

I. Introduction and Problem Statement

Smart-home interaction commonly relies on switches, touchscreens, remote controls, or voice assistants. These interfaces assume particular abilities in reach, speech, or fine motor control. A motion-based interface could provide an additional option for users who can perform comfortable, repeatable wrist movements. It would complement existing accessible controls rather than meet every user's needs. Accessibility benefit must ultimately be established with users, not inferred from classification scores.

An IMU supplies three accelerometer and three gyroscope channels without a camera's dependence on lighting or line of sight. A local classifier could translate a short motion sequence into a symbolic command without transmitting raw sensor traces to a cloud service. However, everyday movements can resemble intentional gestures. A useful interface therefore requires both recognition and a reliable no-command path, together with limits on processing delay and resource use.

The project synopsis proposes a wearable controller, an LSTM baseline, a lightweight alternative, compression, and a receiver for smart-home control. Submission 2 develops that concept into a concise design and a working laptop demonstration. The problem addressed here is: *how can a bounded six-channel processing pipeline translate motion-like input into commands, reject background or uncertain predictions, and expose the evidence still needed for embedded deployment?* The implementation is restricted to a reproducible software pipeline. Wireless links and physical actuation remain proposed extensions.

The demonstration uses synthetic data because a suitable real wrist-command dataset could not be obtained and verified under the available network access. Synthetic streams make preprocessing, tensor shapes, split separation, prediction, and command mapping inspectable. They cannot establish clinical usefulness, generalization to people, or robustness to arbitrary everyday motion. All empirical results in this report refer exclusively to that synthetic software experiment.

II. Focused Literature Survey

The review covers ten distinct papers published in 2020–2022, within the required 2020–2026 interval. Titles, author records, publication years, venues, and publication/preprint identifiers were checked against accessible author or laboratory repositories. Publisher and Scopus endpoints were blocked by the environment's network policy. Consequently, *zero papers have independently verified Scopus coverage in this package*; the faculty requirement of ten verified indexed papers is still outstanding. The accompanying source audit distinguishes publication DOIs from associated preprint DOIs and preserves verification evidence. A DOI or a familiar venue alone is not treated as proof of indexing.

A. Wearable and Inertial Recognition

Zhou *et al.*'s TinyHAR combines local temporal processing, cross-channel interaction, and global temporal modelling to obtain a lightweight activity-recognition model [1]. Abedin *et al.* use cross-channel interaction and temporal attention to improve wearable activity representations in Attend and Discriminate [2]. These works motivate retaining both temporal and multichannel information. Their reported activity-recognition results do not establish performance for this project's command vocabulary or rejection policy.

Saha *et al.*'s TinyOdom addresses inertial navigation with hardware-aware model design and quantization-aware search [3]. It demonstrates the importance of sensor disturbances and target-aware resource constraints in an inertial application. Its navigation task and temporal convolutional models differ from

TABLE I
LITERATURE THEMES AND THEIR IMPLICATIONS FOR THIS PROJECT

Reviewed work	Implication and remaining local question
TinyHAR; Attend and Discriminate [1], [2]	Preserve time and channel structure; command rejection and user transfer need separate evidence.
TinyOdom [3]	Inertial robustness and hardware constraints matter; navigation results do not validate gestures.
MCUNet; MCUNetV2; DORY [4]–[6]	Budget activations and data movement, not just weights; verify recurrent kernels.
Once-for-All; TinyTL; constrained training [7]–[9]	Adapt design to target limits; keep this demonstration fixed and host-trained.
MLPerf Tiny [10]	Separate quality, latency, memory, and energy; do not substitute laptop timing for MCU measurements.

command classification, so its resource or accuracy figures cannot be transferred to the present GRU.

B. Resource-Aware Embedded Learning

MCUNet jointly designs network architecture and an inference engine for resource-limited IoT devices [4]. MCUNetV2 introduces patch-based inference to address peak activation memory [5]. DORY explicitly manages tiling and data transfers when deploying neural networks on low-cost MCUs [6]. These studies show why parameter count alone is inadequate: activation lifetimes, buffers, operator implementations, and the memory hierarchy affect feasibility. Their demonstrated execution paths are not evidence that an arbitrary recurrent graph will run on every MCU.

Once-for-All specializes a trained network for deployment constraints [7]. TinyTL reduces activation-memory demands for on-device adaptation [8], while On-Device Training Under 256KB Memory combines algorithm and runtime design for constrained training [9]. These studies motivate task- and target-specific decisions, but architecture search and on-device training would add unnecessary complexity to the present student demonstration. Training is therefore performed offline on a host, and the selected GRU has a fixed, small architecture.

MLPerf Tiny treats embedded inference quality, latency, and energy as separate benchmarking dimensions [10]. That distinction guides the present evaluation: synthetic classification quality and host timing are reported, while MCU memory, MCU latency, and power remain unmeasured. Table I connects the reviewed themes to the local design gap; it does not claim an exhaustive survey or that earlier authors omitted every listed issue.

III. RESEARCH GAP, OBJECTIVES, AND CONTRIBUTIONS

The reviewed literature supports temporal modelling, target-aware execution, and multidimensional evaluation. It does not supply evidence for this project's particular gesture-to-command pipeline. The specific gap addressed in this mini-project is an *inspectable integration* of six-channel preprocessing, fixed-size windowing, a compact temporal classifier, background handling, and command rejection, with clearly separated software and hardware claims. This is an application-specific engineering gap, not a claim that wearable gesture control or compact recurrent models are new research topics.

The objectives are to: (1) implement a repeatable sensor-like input-to-command demonstration; (2) prevent recording/window leakage and select a checkpoint without using test labels; (3) expose a no-command path; and (4) document resource estimates and the additional checks required for MCU deployment. The contributions are runnable Python code with a small saved model, an explicit split and evaluation record, an embedded-oriented architecture, and a bounded account of what the results establish. No novel neural architecture or experimentally proven assistive benefit is claimed.

IV. PROPOSED METHODOLOGY AND ARCHITECTURE

Fig. 1 distinguishes the implemented laptop path from future physical components. The software processes six nominal channels, $\mathbf{x}_t = [a_x, a_y, a_z, \omega_x, \omega_y, \omega_z]^\mathsf{T}$. A future MCU could acquire equivalent channels using a timer or IMU data-ready interrupt and an I^{2}C/SPI driver, timestamping samples and detecting missed samples or sensor saturation. No such acquisition hardware has been built or selected.

A. Acquisition, Filtering, and Windowing

The chosen software settings are a nominal 50 Hz sample rate, 64 samples per window, and a stride of 32 samples. A window represents 1.28 s of generated signal, with a new window opportunity every 0.64 s. These are design timings, not measured response latency. A future fixed-size ring buffer would retain the most recent samples; the host demonstration forms the equivalent windows from finite arrays rather than implementing an interrupt-driven driver.

A causal first-order filter is independently reset at each recording boundary:

$$\mathbf{y}_0 = \mathbf{x}_0, \qquad \mathbf{y}_t = 0.35\mathbf{x}_t + 0.65\mathbf{y}_{t-1}. \tag{1}$$

This simple filter smooths noise but adds delay and is not a substitute for sensor-side anti-alias filtering. Each channel is subsequently standardized using means and standard deviations computed from *unique filtered training samples only*. These constants are frozen for validation, testing, and saved-model inference. The model input is the full 64×6 sequence; feature extraction is learned inside the recurrent layer. No additional handcrafted classifier or feature-selection branch is introduced.

B. Compact GRU and Analytical LSTM Reference

The implemented classifier is a single-layer, unidirectional GRU with 16 hidden units and a linear four-class output head. A softmax converts logits to class probabilities. The classes are background, swipe left, swipe right, and wave; they map to `NO_COMMAND`, `LIGHT_OFF`, `LIGHT_ON`, and `FAN_TOGGLE`, respectively. This vocabulary is a deliberately reduced software example, not a validated human-gesture

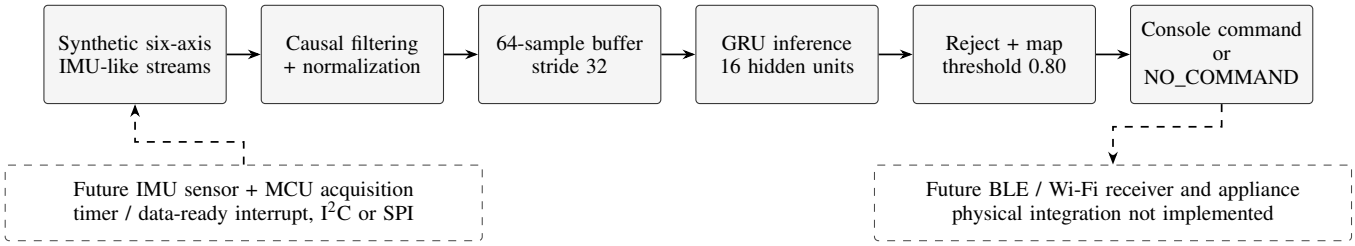


Fig. 1. Sensor-to-command architecture. Solid blocks are exercised by the laptop software using synthetic input. Dashed components are proposed physical extensions only; no sensor, MCU, receiver, wireless link, or appliance is required to run this demonstration.

taxonomy. A hold is excluded because stillness would require an explicit interaction context.

Each window starts from a zero recurrent state and is classified from its final hidden vector. This gives bounded, independent inference calls, though overlapping windows recompute some samples. The host implementation uses PyTorch's GRU convention with separate input and recurrent bias vectors. With $d = 6$, $h = 16$, and $C = 4$, its parameter count is

$$P_{\mathrm{GRU}} = 3h(d + h + 2) + (h + 1)C. \tag{2}$$

This gives 1220 parameters. An otherwise comparable 32-unit LSTM would have $4h(d+h+2)+(h+1)C = 5252$ parameters under the same double-bias convention. This is an *analytical comparison only*: no LSTM has been trained or benchmarked. The smaller GRU payload does not establish equal accuracy or a measured speedup.

C. Rejection and Command Mapping

The decision rule is

$$c = \begin{cases} \mathrm{map}(k), & k \neq 0,\ p_k \geq 0.80, \\ \mathrm{NO_COMMAND}, & \text{otherwise}, \end{cases} \tag{3}$$

where $k = \arg\max_j p_j$ and class 0 is background. The threshold is declared before testing rather than tuned on test performance. Background samples therefore generate no command even at high confidence. Low confidence also suppresses a command, but a softmax threshold is not a guaranteed detector of unfamiliar motion. The console prints window decisions only. Temporal persistence, cooldown, acknowledgements, and secure physical actuation are future integration tasks; this program does not claim to eliminate duplicate commands in a continuous home environment.

D. Resource and Quantization Constraints

At deployment, flash must accommodate model data, inference kernels, firmware, and constants. Peak RAM includes the ring buffer, normalized input, recurrent states, scratch arena, stack, and any radio buffers. One 64×6 window contains 768 bytes of 16-bit samples or 1536 bytes of FP32 input; these are payload calculations, not total MCU RAM. The implemented FP32 model has 4880 bytes of parameter payload, excluding host runtime and serialization overhead.

Optional 8-bit post-training quantization could map $r \approx s(q - z)$ using scale s and zero point z, calibrated with training data only. It has not been implemented. Integer GRU gates, nonlinearities, recurrent state handling, and conversion consistency would need verification in a chosen runtime. Wider biases or states, metadata, fallback operations, and scratch storage can limit savings; quantization can also reduce accuracy. Pruning is not part of the demonstration, and introducing zeros would not by itself reduce dense storage or latency. Arduino Nano 33 BLE, ESP32, and LPC1768-based systems are examples with different MCU, memory, peripheral, and runtime constraints, not interchangeable or verified deployment targets.

V. Software Demonstration and Pseudocode

A. Synthetic Signal Generation and Data Separation

The generator creates independent 192-sample recordings with six channels in nominal acceleration and angular-velocity units. Gravity is represented by an approximately constant a_z offset. Background contains low-amplitude noise and drift. Left and right examples include opposite-sign stylized angular-velocity pulses with correlated acceleration; wave examples use oscillatory acceleration and angular velocity. Frequency, phase, gain, offset, and noise vary by recording. These convenient class formulas deliberately define a learnable toy problem; they are not a validated physical simulator and do not reproduce the diversity of real wrist movement.

The fixed seed is 42. Per class, 24 recordings are assigned to training, 8 to validation, and 10 to testing *before filtering and windowing*. Each recording yields five windows. The resulting partitions contain 480, 160, and 200 windows from 96, 32, and 40 distinct recordings, respectively. Recording IDs and generator seeds are stored in a manifest. No overlapping window or recording crosses partitions. This tests separation of synthetic recordings, not transfer between real participants.

B. Training and Reproduction

Training uses FP32 CPU execution, Adam with learning rate 0.004, batch size 64, and categorical cross-entropy for 30 epochs. The checkpoint with the lowest validation cross-entropy is retained. Test labels are used only after this selection. The same fixed filtering and normalization pipeline is applied to every split. Fig. 2 summarizes the executed process; Fig. 3 presents its central logic as pseudocode.

The supplied checkpoint allows an immediate rerun; a training flag rebuilds it. The package also records Python, NumPy,

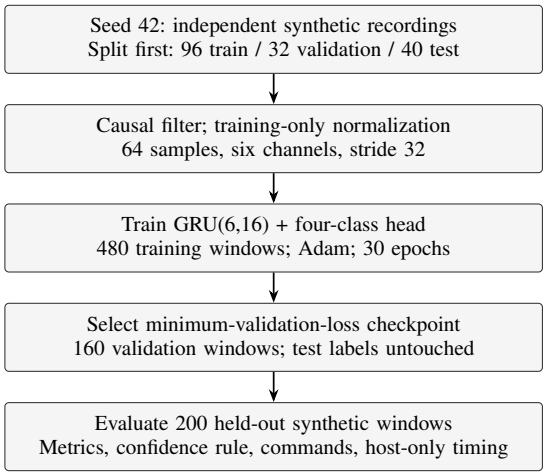


Fig. 2. Executed synthetic-data workflow. Split membership is assigned at recording level before windows exist. Only training data fit normalization and model weights; validation selects the checkpoint. Test data measure the final software pipeline. Quantization and MCU deployment are outside this executed path.

Algorithm 1: Synthetic input to command

1. Set seed 42; assign independent recording IDs to splits.
2. Generate six-axis signals for each labelled recording.
3. Causally filter each recording; reset filter at boundaries.
4. Fit normalization constants from training samples only.
5. Normalize all splits with those frozen constants.
6. Form 64-sample windows, stride 32, within each recording.
7. Train GRU; retain lowest-validation-loss checkpoint.
8. For each held-out test window:
   $p \leftarrow \mathrm{softmax}(\mathrm{GRU}(\mathrm{window}))$.
   $k \leftarrow \arg\max p$.
   If k is background or $p_k < 0.80$:
      $c \leftarrow$ NO_COMMAND.
   Else: $c \leftarrow \mathrm{command_map}[k]$.
   Print label, confidence, and c.
9. Save metrics, split manifest, model, and example output.

Fig. 3. Pseudocode of the implemented demonstration.

and PyTorch versions and the host environment. Seven automated checks exercise group separation, training-only normalization, causality, malformed input rejection, command rules, a known confusion matrix, and model/window dimensions. These tests check implementation properties; passing them does not validate assistive use.

VI. Results, Discussion, and Limitations

A. Actual Synthetic Software Results

Table II reports the saved run, not hypothetical values. Precision, recall, and F1 are computed from argmax predictions before threshold-based command rejection; macro averages include all four classes with equal class weighting. Accuracy is $\sum_k TP_k/n$, precision is $TP_k/(TP_k + FP_k)$, recall is $TP_k/(TP_k + FN_k)$, and $F1_k$ is the harmonic mean of precision and recall. A zero denominator is assigned zero in the implementation. All test classes have support 50.

The confusion matrix is diagonal with 50 correct windows per class. The first held-out examples print background $\rightarrow$ `NO_COMMAND`, left $\rightarrow$ `LIGHT_OFF`, right $\rightarrow$ `LIGHT_ON`, and wave $\rightarrow$ `FAN_TOGGLE`. Actual confidences and commands are preserved in the output transcript. All 150 accepted gesture-window commands were correct on this generated test set. A separate unit check verifies that a non-background maximum probability of 0.70 is suppressed at the 0.80 threshold.

TABLE II
Recorded Synthetic Test and Host Results

Metric	Recorded value
Held-out synthetic windows	200 (40 recordings)
Accuracy	100.0%
Macro precision / recall / F1	1.000 / 1.000 / 1.000
Accepted command windows	150 of 150 gesture windows
Commands on synthetic background	0 of 50 windows
Host median / p95 model latency	0.668 / 0.949 ms
FP32 parameters / weight payload	1220 / 4880 bytes
Saved checkpoint size	7720 bytes

Perfect toy-data scores are unsurprising because the generator creates simple, separable class patterns and train/test recordings share those formulas. They demonstrate that the pipeline can learn and map those patterns; they provide *no estimate of real-world recognition accuracy*. The synthetic background test also does not establish a real false-activation rate per hour. There is no real-user dataset, external evaluation population, or physical gesture ground truth.

B. Timing and Embedded Interpretation

Model timing uses a prepared, batch-one input, CPU inference mode, one PyTorch thread, ten warm-up calls, and 100 timed calls. The median and p95 in Table II describe this host run and can vary on another laptop. They exclude acquisition, window filling, filtering, confidence mapping, and output transport. They are neither end-to-end response delay nor MCU latency. Host framework overhead and optimized CPU kernels prevent direct extrapolation to a microcontroller.

The weight payload is computed from actual FP32 tensors; checkpoint size additionally includes serialization and saved constants. Neither is a measurement of firmware flash or peak MCU RAM. A future target evaluation would inspect a linker map, inference-arena allocation, stack high-water marks, and concurrent buffers. Power would require a defined supply boundary and voltage/current measurement; energy would be $E = \int V(t)I(t)\,dt$. None of those hardware measurements has been performed, and battery life is not estimated.

C. Limitations and Future Work

The principal limitation is the synthetic signal distribution: real motion introduces variable placement, orientation, tremor, fatigue, incomplete gestures, and incidental activities absent here. Background is not representative of the open set of everyday movements, and confidence is not calibrated as a probability of safe action. Before physical control, the design would need real labelled recordings, participant/session-separated evaluation, continuous-stream event matching, false activations per hour, duplicate suppression, and user-centred accessibility assessment.

The LSTM comparison, quantization, target selection, wireless receiver, and appliance interface remain analytical or future work. Literature indexing verification and an institution-approved similarity report are separate outstanding submission checks. Future development should first validate a suitable real-data protocol and MCU/runtime operator support, then measure on-target resource use. Additional applications or dashboards are unnecessary for the present scope.

VII. Conclusion

Submission 2 delivers an executable six-channel synthetic processing and command-generation demonstration alongside an embedded-oriented design. A compact GRU, training-only normalization, recording-level split, background class, and confidence rejection provide a clear and reproducible software example. Actual synthetic metrics and host timings are reported with explicit boundaries; no result is presented as a real-user or MCU benchmark. The literature supports attention to temporal structure, memory behaviour, and evaluation scope while leaving indexing confirmation outstanding. The package advances the synopsis from a theoretical architecture to a working software pipeline without claiming hardware deployment or validated assistive performance.

References

[1] Y. Zhou, H. Zhao, Y. Huang, M. Hefenbrock, T. Riedel, and M. Beigl, "TinyHAR: A Lightweight Deep Learning Model Designed for Human Activity Recognition," in *Proceedings of the International Symposium on Wearable Computers*, 2022. [Online]. Available: https://doi.org/10.1145/3544794.3558467

[2] A. Abedin, M. Ehsanpour, Q. Shi, H. Rezatofighi, and D. C. Ranasinghe, "Attend and Discriminate: Beyond the State-of-the-Art for Human Activity Recognition using Wearable Sensors," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 2021. [Online]. Available: https://doi.org/10.1145/3448083

[3] S. S. Saha, S. S. Sandha, L. A. Garcia, and M. Srivastava, "TinyOdom: Hardware-Aware Efficient Neural Inertial Navigation," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 2022. [Online]. Available: https://doi.org/10.1145/3534594

[4] J. Lin, W.-M. Chen, Y. Lin *et al.*, "MCUNet: Tiny Deep Learning on IoT Devices," in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: https://doi.org/10.48550/arXiv.2007.10319

[5] J. Lin, W.-M. Chen, H. Cai, C. Gan, and S. Han, "MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning," in *Advances in Neural Information Processing Systems*, 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2110.15352

[6] A. Burrello, A. Garofalo, N. Bruschi, G. Tagliavini, D. Rossi, and F. Conti, "DORY: Automatic End-to-End Deployment of Real-World DNNs on Low-Cost IoT MCUs," *IEEE Transactions on Computers*, 2021. [Online]. Available: https://doi.org/10.1109/TC.2021.3066883

[7] H. Cai, C. Gan, T. Wang, Z. Zhang, and S. Han, "Once-for-All: Train One Network and Specialize it for Efficient Deployment," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://doi.org/10.48550/arXiv.1908.09791

[8] H. Cai, C. Gan, L. Zhu, and S. Han, "TinyTL: Reduce Activations, Not Trainable Parameters for Efficient On-Device Learning," in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: https://doi.org/10.48550/arXiv.2007.11622

[9] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "On-Device Training Under 256KB Memory," in *Advances in Neural Information Processing Systems*, 2022. [Online]. Available: https://doi.org/10.48550/arXiv.2206.15472

[10] C. Banbury, V. J. Reddi, P. Torelli *et al.*, "MLPerf Tiny Benchmark," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. [Online]. Available: https://doi.org/10.48550/arXiv.2106.07597